

Assignment No 3

Aim : Implement heuristic-based search algorithms such as A* and Greedy Best- First Search, and evaluate their effectiveness in finding optimal or near- optimal solutions

Part A

To implement the *A Search Algorithm** and find the shortest (optimal) path between a start node and a goal node using heuristic information.

Objective

- To understand heuristic search techniques.
- To implement the A* Search Algorithm.
- To find the optimal path between two nodes.
- To understand the role of heuristic functions in Artificial Intelligence.

Theory

A* (pronounced **A-Star**) is one of the most popular informed search algorithms used in Artificial Intelligence for pathfinding and graph traversal. It combines the advantages of **Uniform Cost Search** and **Greedy Best-First Search** by considering both:

- The actual cost from the start node to the current node.
- The estimated cost from the current node to the goal.

Using these two values, A* efficiently finds the shortest possible path while minimizing unnecessary exploration.

Evaluation Function

The evaluation function used in A* is:

$$f(n)=g(n)+h(n)$$

Where:

- **g(n)** = Actual cost from the start node to node *n*.
- **h(n)** = Estimated cost (heuristic) from node *n* to the goal.

- **$f(n)$** = Estimated total cost of the path through node n .

The algorithm always expands the node with the **smallest $f(n)$** value.

Working of A*

1. Start from the initial node.
2. Calculate the **$g(n)$, $h(n)$, and $f(n)$** values.
3. Select the node with the smallest **$f(n)$** .
4. Expand its neighboring nodes.
5. Update costs if a shorter path is found.
6. Repeat until the goal node is reached.

Advantages

- Finds the shortest path.
- Complete and optimal (with an admissible heuristic).
- Faster than uninformed search algorithms.
- Widely used in real-world applications.

Disadvantages

- Uses more memory.
- Performance depends on the quality of the heuristic function.

Algorithm

1. Create an open list (priority queue).
2. Insert the start node.
3. Set the cost of the start node to zero.
4. While the open list is not empty:
 - Remove the node with the lowest **$f(n)$** .
 - If it is the goal node, stop.
 - Expand all neighboring nodes.
 - Update their costs.

- Insert unexplored nodes into the open list.
5. Display the shortest path and total cost

```
from queue import PriorityQueue
```

```
graph = {  
    'A': [('B', 1), ('C', 4)],  
    'B': [('D', 2), ('E', 5)],  
    'C': [('F', 3)],  
    'D': [('G', 1)],  
    'E': [('G', 2)],  
    'F': [('G', 5)],  
    'G': []  
}
```

```
heuristic = {  
    'A': 7,  
    'B': 6,  
    'C': 4,  
    'D': 2,  
    'E': 2,  
    'F': 4,  
    'G': 0  
}
```

```
def a_star(start, goal):  
    pq = PriorityQueue()  
    pq.put((0, start))  
  
    g_cost = {start: 0}  
    parent = {}  
  
    while not pq.empty():  
        _, current = pq.get()  
  
        if current == goal:  
            path = []
```

```

while current != start:
    path.append(current)
    current = parent[current]
path.append(start)
path.reverse()
return path, g_cost[goal]

for neighbor, cost in graph[current]:
    new_cost = g_cost[current] + cost

    if neighbor not in g_cost or new_cost < g_cost[neighbor]:
        g_cost[neighbor] = new_cost
        priority = new_cost + heuristic[neighbor]
        pq.put((priority, neighbor))
        parent[neighbor] = current

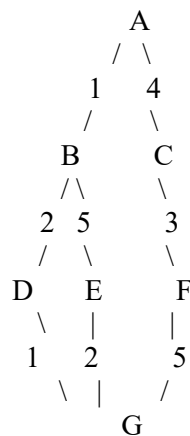
return None

path, cost = a_star('A', 'G')

print("Shortest Path:", path)
print("Total Cost:", cost)

```

Sample Output



Shortest Path: ['A', 'B', 'D', 'G']

Total Cost: 4

Conclusion

The A* (A-Star) Search Algorithm was successfully implemented and tested for finding the shortest path between a start node and a goal node. By combining the actual path cost **g(n)** with the heuristic

estimate $h(n)$ using the evaluation function $f(n) = g(n) + h(n)$, the algorithm efficiently selected the most promising nodes during the search process. The results demonstrated that A* is both **complete** and **optimal** when an admissible heuristic is used, making it more effective than uninformed search algorithms for pathfinding problems. Due to its ability to find the optimal solution with reduced search effort, A* is widely used in applications such as GPS navigation, robotics, game AI, and network routing.

Aim

To implement the **Greedy Best-First Search (GBFS)** algorithm and evaluate its effectiveness in finding optimal or near-optimal solutions using heuristic information.

Objectives

- To understand the concept of heuristic search.
 - To implement the Greedy Best-First Search algorithm.
 - To analyze the role of heuristic functions in search problems.
 - To evaluate the performance of GBFS in terms of efficiency and solution quality.
 - To compare its characteristics with other search algorithms.
-

Theory

Introduction

Greedy Best-First Search (GBFS) is an **informed (heuristic) search algorithm** used in Artificial Intelligence for solving pathfinding and graph traversal problems. Unlike uninformed search algorithms, GBFS uses a heuristic function to estimate how close each node is to the goal and always selects the node that appears to be the most promising.

The algorithm is called **greedy** because it always makes the locally optimal choice by selecting the node with the lowest heuristic value without considering the total path cost.

Heuristic Function

The heuristic function is represented as:

$$h(n)$$

where:

- **n** = Current node
- **h(n)** = Estimated cost from node n to the goal

The evaluation function for Greedy Best-First Search is:

$$f(n)=h(n)$$

The node with the smallest heuristic value is expanded first.

Working Principle

1. Start from the source node.
2. Place the source node in a priority queue.
3. Select the node having the minimum heuristic value.
4. If the selected node is the goal, stop.
5. Otherwise, expand all neighboring nodes.
6. Insert the neighbors into the priority queue based on their heuristic values.
7. Repeat the process until the goal is reached or no path exists.

Algorithm

1. Initialize an empty priority queue.
2. Insert the start node into the queue.
3. Mark the start node as visited.
4. While the priority queue is not empty:
 - Remove the node with the smallest heuristic value.
 - If it is the goal node, reconstruct and display the path.
 - Otherwise, visit all unvisited neighboring nodes.
 - Add neighbors to the priority queue according to their heuristic values.
5. End.

```
from queue import PriorityQueue

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': ['G'],
    'E': ['G'],
    'F': ['G'],
    'G': []
}

heuristic = {
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 2,
    'E': 2,
    'F': 4,
    'G': 0
}

def greedy_best_first(start, goal):
    pq = PriorityQueue()
    pq.put((heuristic[start], start))

    visited = set()
    parent = {}

    while not pq.empty():
        _, current = pq.get()
```

```
    if current == goal:
        path = []
        while current != start:
            path.append(current)
            current = parent[current]
        path.append(start)
        return path[::-1]

    if current not in visited:
        visited.add(current)

        for neighbor in graph[current]:
            if neighbor not in visited:
                parent[neighbor] = current
                pq.put((heuristic[neighbor], neighbor))

    return None

path = greedy_best_first('A', 'G')

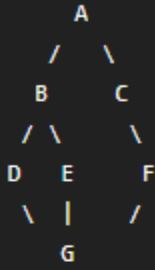
print("Path Found:")
print(path)
```

Sample Output

Path Found:

['A', 'C', 'F', 'G']

Graph



Heuristic Values

Node	$h(n)$
A	7
B	6
C	4
D	2
E	2
F	4
G	0

Conclusion

The **Greedy Best-First Search (GBFS)** algorithm was successfully implemented and evaluated for finding a path between a start node and a goal node. The algorithm selects the next node based solely on the heuristic value $h(n)$, which estimates the distance to the goal. This approach enables GBFS to reach the destination quickly by exploring the most promising nodes first. However, since it does not consider the actual path cost, it may produce a **near-optimal** rather than the **optimal** solution. The evaluation showed that Greedy Best-First Search is generally faster and requires less computation than optimal search algorithms like A*, making it suitable for applications where speed is more important than guaranteed shortest paths. Overall, GBFS is an efficient heuristic search technique for solving large search problems when approximate solutions are acceptable.

